

IFT436 - Algorithmes et structures de données

Devoir #2

— Question 1 un peu de grand-O —

- (a) (i) Oui
(ii) Oui
(iii) Non
(iv) Oui
(v) Non

(b) i

$$f(n) = n^4 + 20n^2$$

Montrer que $f(n) \in \Omega(n^4)$

Pour $n \geq 1$,

$$f(n) = n^4 + 20n^2 \geq n^4.$$

On peut donc choisir $c = 1$, $n_0 = 1$.

$$\boxed{f(n) \in \Omega(n^4)}.$$

Montrer que $f(n) \in O(n^4)$

Pour $n \geq 1$,

$$n^2 \leq n^4.$$

$$f(n) = n^4 + 20n^2 \leq n^4 + 20n^4 = 21n^4.$$

On peut donc choisir $c = 21$, $n_0 = 1$.

$$\boxed{f(n) \in O(n^4)}.$$

Puisque $f(n) \in O(n^4)$ et $f(n) \in \Omega(n^4) \implies \boxed{f(n) \in \Theta(n^4)}.$

$$g(n) = \left(\frac{n^2}{1000} + 1 \right)^2 = \frac{n^4}{10^6} + \frac{n^2}{500} + 1.$$

Montrer que $g(n) \in \Omega(n^4)$

Pour $n \geq 1$,

$$g(n) = \frac{n^4}{10^6} + \frac{n^2}{500} + 1 \geq \frac{n^4}{10^6}.$$

On peut donc choisir $c = \frac{1}{10^6}$, $n_0 = 1$.

$$\boxed{g(n) \in \Omega(n^4)}.$$

Montrer que $g(n) \in O(n^4)$

Pour $n \geq 1$,

$$n^2 \leq n^4 \quad \text{et} \quad 1 \leq n^4.$$

$$g(n) = \frac{n^4}{10^6} + \frac{n^2}{500} + 1$$

$$\leq \frac{n^4}{10^6} + \frac{n^4}{500} + n^4.$$

$$g(n) \leq \underbrace{\left(\frac{1}{10^6} + \frac{1}{500} + 1 \right)}_k n^4.$$

On peut donc choisir $c = k$ $n_0 = 1$

$$\boxed{g(n) \in O(n^4)}.$$

Puisque $g(n) \in O(n^4)$ et $g(n) \in \Omega(n^4) \implies \boxed{g(n) \in \Theta(n^4)}.$

Conclusion :

$$\boxed{f(n) \in \Theta(n^4)} \quad \text{et} \quad \boxed{g(n) \in \Theta(n^4)} \implies \boxed{f(n) \in \Theta(g(n))}.$$

iii

$$f(n) = n! = n(n-1)(n-2) \cdots 2 \cdot 1$$

$$g(n) = (2n)! = (2n)(2n-1)(2n-2) \cdots (n+1)n(n-1) \cdots 2 \cdot 1$$

On reconnaît alors les facteurs de $n!$:

$$(2n)! = \underbrace{(2n)(2n-1)(2n-2) \cdots (n+1)}_{\text{termes en } n} \underbrace{n(n-1) \cdots 2 \cdot 1}_{n!}$$

Donc

$$(2n)! = n!(n+1)(n+2) \cdots (2n)$$

Ainsi,

$$\frac{(2n)!}{n!} = (n+1)(n+2) \cdots (2n)$$

Il n'existe donc aucune constante $c > 0$ telle que

$$f(n) \leq c g(n)$$

pour tout $n \geq n_0$.

Par définition de O ,

$$f(n) \notin O(g(n)).$$

Question 2 le tri sélection

Susposons qu'on utilise un tableau de hashage qui fait tout en temps "constant" $O(1)$.*.

Pour trouver le minimum d'un tableau non trié le mieux qu'on puisse faire est un parcours séquentiel simple en un seul passage (complexité $O(n)$). Il est impossible de faire mieux en termes de complexité algorithmique car il faut nécessairement examiner chaque élément au moins une fois.

Dans le pire cas, les tableau T est parfaitement désordonné, dans ce cas, trouver le min pour la première iteration prendra un temps n . La second prendra un temps $n-1$ puisque nous auront retiré un élément. Puis $(n-3)$ etc. la i^{iem} iteration prendra un temps $n - (i - 1)$.

Le temps total est donc $t(n) = n + (n - 1) + (n - 2) + (n - 3) + (n - 4)$ On reconnait la formule de la somme des n premiers entiers.

$$t(n) = n + (n - 1) + (n - 2) + (n - 3) + (n - 4) = \frac{n(n+1)}{2}$$

$$t(n) = \frac{n^2 + n}{2}$$

Pour montrer que $t(n) \in \Theta(n^2)$, on veut montrer que $t(n) \in O(n^2)$ et que $t(n) \in \Omega(n^2)$.

Pour $n \geq 1$, comme le terme dominant est $\frac{1}{2}n^2$,

$$\frac{n^2}{2} \leq t(n) \quad \text{et} \quad \frac{n^2 + n}{2} \leq n^2.$$

Ainsi,

$$t(n) \in O(n^2), \text{ avec } c = 1 \quad \text{et} \quad t(n) \in \Omega(n^2), \text{ avec } c = \frac{1}{2} \implies t(n) \in \Theta(n^2).$$

On rappelle que les opérations d'ajout et de suppression étant en $O(1)$ avec la table de hashage, elles ne changent pas la complexité totale.

Question 3 Détection de collision 1D

- (a) Pour chaque point $p \in P$, l'algorithme parcourt tous les intervalles $(a, b) \in I$. Il effectue donc $|P| = m$ parcours contenant chacun $|I| = n$ tests.
si on suppose que chaque test $a \leq p \leq b$ prend un temps constant $O(1)$.
Ainsi, le nombre total d'opérations est proportionnel à

$$|P| \times |I| = m \times n.$$

Donc la complexité en temps de l'algorithme naïf est

$$\boxed{O(mn)}.$$

- (b) On suppose que $|P| = |I| = n$ et que tous les intervalles sont positifs.

L'idée serait de d'abord trier les points et les intervalles selon leur position. Pour chaque point p , on considère toutes les intervalles dont la borne gauche a est inférieure ou égale à p . Parmi ces intervalles, il suffit de conserver celui dont la borne droite b est la plus grande. En effet, si ce plus grand b est inférieur à p , aucun des intervalles considérés ne peut contenir p . À l'inverse, si $\max_b \geq p$, alors l'intervalle correspondant contient p . Comme les intervalles sont triés selon leur borne gauche a et que les points sont triés par ordre croissant, l'indice j avance toujours dans la même direction. Lorsqu'un intervalle est examiné pour un point p , il n'est donc jamais nécessaire de le revoir pour les points suivants, puisque ceux-ci sont plus grands ou égaux à p . Ainsi, chaque intervalle est examiné au plus une seule fois pendant toute l'exécution de l'algorithme. On effectue essentiellement un "swipe" uniquement sur les intervalles ou les points peuvent possiblement être.

Collision(P, I):

```
trier P par ordre croissant
trier I par borne gauche a croissante

j <- 1
max_b <- 0
intervalle_max <- null

pour chaque p dans P, dans l'ordre croissant:

    tant que j <= |I| et I[j].a <= p:
        si I[j].b > max_b:
            max_b <- I[j].b
            intervalle_max <- I[j]
        j <- j + 1

    si max_b >= p:
        retourner (p, intervalle_max)

retourner null
```

Complexité :

Le tri de P coûte

$$O(n \log n)$$

et le tri de I coûte également

$$O(n \log n).$$

La boucle parcourt chaque point et chaque intervalle au plus une fois, donc elle coûte

$$O(n + n) = O(2n) = O(n).$$

Ainsi l'algorithme prend un temps $O()$ total de,

$$O(n \log n) + O(n \log n) + O(n) = \boxed{O(n \log n)}.$$

Question 4 Détection de collision 1D

- (a) L'idée est d'effectuer un parcours (BFS) en utilisant un dictionnaire **vis** qui associe à chaque sommet visité son parent. Lorsqu'un nouveau sommet est découvert, on l'ajoute à **vis** avec le sommet courant comme parent. Si on rencontre un sommet w qui est déjà dans **vis** et qui n'est pas le parent du sommet courant z , alors l'arête (z, w) ferme un cycle. On utilise ensuite les parents contenus dans **vis** pour reconstruire les sommets de ce cycle.

`Cycle(G = (V,E)):`

```
vis <- dict(sommet, parent)

pour chaque sommet v_0 dans V:

    si v_0 n'est pas dans vis:

        file <- (v_0)
        vis.insert(v_0, null)

        tant que !file.empty():

            z <- file.pop_front()

            pour chaque w dans {voisins de z}:

                si w n'est pas dans vis:

                    file.push_back(w)
                    vis.insert(w, z)

                sinon si w != vis(z):

                    retourner ReconstruireCycle(z, w, vis) %voir ci bas

retourner null
```

Lorsqu'on appelle `ReconstruireCycle`, les sommets z et w sont déjà dans `vis` et l'arête (z, w) ne correspond pas à une relation parent-enfant. Il existe donc déjà dans l'arbre construit par le BFS un chemin reliant z et w , et l'arête (z, w) vient fermer ce chemin pour former un cycle.

Pour reconstruire ce cycle, on remonte d'abord les parents de z jusqu'à la "racine" (le v_0 choisi au départ) et on conserve les sommets rencontrés dans `chemin_z`. On remonte ensuite les parents de w jusqu'à rencontrer le premier sommet appartenant aussi à `chemin_z`; ce sommet est leur premier ancêtre commun. Les deux chemins entre cet ancêtre et z et w , auxquels on ajoute l'arête (z, w) , forment alors le cycle recherché.

`ReconstruireCycle(z, w, vis):`

```
chemin_z <- []
chemin_w <- []

x <- z
tant que x != null:
    chemin_z.push_back(x)
    x <- vis(x)

x <- w
tant que x n'est pas dans chemin_z:
    chemin_w.push_front(x)
    x <- vis(x)

ancetre <- x

C <- []

x <- z
tant que x != ancetre:
    C.push_front(x)
    x <- vis(x)

C.push_front(ancetre)

pour chaque x dans chemin_w:
    C.push_back(x)

C.push_back(z)

retourner C
```

- (b) Un arbre contenant $|V|$ sommets possède exactement $|V| - 1$ arêtes. Puisqu'un graphe unicycle doit devenir un arbre après le retrait d'une seule arête, il doit donc posséder exactement $|V|$ arêtes. De plus, le graphe doit être connexe, puisqu'enlever une arête ne peut pas connecter deux composantes séparées. Ainsi, il suffit de vérifier que G est connexe et que $|E| = |V|$.

`Unicycle(G = (V,E)):`

```
si |E| != |V|:
    retourner false
```

```

    si !Connectivite(G):
        retourner false

```

```

    retourner true

```

Si on suppose que $|E|$ et $|V|$ sont conservés dans $G(V, E)$ alors la comparaison entre $|E|$ et $|V|$ se fait en temps constant (*i*). Sinon on doit compter les éléments de $(|V|)$ et $(|E|)$ en les parcourant, ce serait respectivement $O(|V|)$ et $O(|E|)$.

Connectivite($G = (V, E)$):

```

    choisir v_0 dans V
    vis <- BFS(G, v_0)

```

```

    retourner |vis| == |V|

```

La fonction **Connectivite** utilise le BFS vu en classe, qui possède une complexité de $O(n \log n)$. Cette opération domine donc la complexité de l'algorithme.

Ainsi, la complexité totale est

$$\boxed{O(n \log n)}.$$